

COMP3013 DBMS Lab 8

United International College

NULL

- The domain of each data type contains a special value – **NULL**.
- **NULL** means the value is **unknown** or does **not exist**.
- NULL values can participate
 - calculations in the arithmetic expressions,
 - aggregation functions,
 - comparisons, and
 - predicates (logical test) in the WHERE clause.

Arithmetic and Atomic Predicate

- To check whether a value of an attribute is NULL (atomic predicate)

`WHERE attribute IS NULL`

- For example,

`SELECT * FROM staff WHERE picture IS NULL`

- Any arithmetic expression which has NULL values is evaluated as NULL.
- For example, this query is same as the above.

`SELECT * FROM staff WHERE (picture+1) IS NULL`

Aggregation

- Different aggregation functions treat NULL values differently.
- If NULL values and non-NULL values are mixed,
 - all aggregation functions ignore NULL values.
- If the group only consists of NULL values,
 - count returns 0;
 - other aggregation functions return NULL.
- Try the follow queries and see the outcome. One staff has rating NULL.
`SELECT COUNT(staff_rating) FROM staff`
`SELECT MIN(staff_rating) FROM staff`
`SELECT staff_id,COUNT(staff_rating) FROM staff GROUP BY staff_id`
`SELECT staff_id,MIN(staff_rating) FROM staff GROUP BY staff_id`

Comparison and Compound Predicates

- The logic with NULL values has three constants:
TRUE, FALSE, and **UNKNOWN**
- A comparison with NULL values is evaluated as UNKNOWN.
- To check whether a logical constant is UNKNOWN, use **IS UNKNOWN**.
- For example, try the query and check the outcome.

```
SELECT staff_id FROM staff WHERE (staff_rating>5) IS UNKNOWN
```

Comparison and Compound Predicates

- In the compound predicates, (with logical connectives: AND, OR, and NOT)
 - OR: (unknown **OR** true) = true
(unknown **OR** false) = unknown
(unknown **OR** unknown) = unknown
 - AND: (true **AND** unknown) = unknown
(false **AND** unknown) = false
(unknown **AND** unknown) = unknown
 - NOT: (**NOT** unknown) = unknown
- If a logical test in WHERE is eventually equal to UNKNOWN, the result of the test is treated as FALSE.

Example

- Try the examples and explain the outcomes.

```
SELECT staff_id FROM staff WHERE (staff_rating>5 OR true);  
SELECT staff_id FROM staff WHERE (staff_rating>5 AND true);  
SELECT staff_id FROM staff WHERE (NOT staff_rating>5);
```

Subquery

- A **subquery** is a query (SELECT...FROM...WHERE) inside another query.
- Subqueries can be in all of SELECT, FROM, and WHERE clauses.
- This lab introduces subqueries in the FROM clause.
- In the FROM clause, users need to indicate some tables.
- A “SELECT...FROM...WHERE...” query returns a temporary table. (The type is matched.)

Subquery

- For example, find the id of the actors who have played some films of rating “PG-13”.

```
SELECT actor_id
FROM film_actor NATURAL JOIN (
    SELECT film_id
    FROM film
    WHERE rating = 'PG-13'
) AS temp
```

- The keyword `AS` renames the temporary table returned by the `subquery` as `'temp'`.

Subquery

- The previous example can also be done without a subquery.

Query 1:

```
SELECT actor_id
FROM film_actor NATURAL JOIN (
    SELECT film_id
    FROM film
    WHERE rating = 'PG-13'
) AS temp
```

Query 2:

```
SELECT actor_id
FROM film_actor JOIN film
    USING(film_id)
WHERE rating='PG-13'
```

- What is the difference between these two approaches?
- Which one is more efficient?
- (Think about the procedure when queries are executed by the system, **about optimization.**)

Example

- Find the name of the customers who live in China.

those
customers

```
SELECT first_name, last_name
FROM customer, (
  SELECT address_id
  FROM address, (
    SELECT city_id
    FROM city, (
      SELECT country_id
      FROM country
      WHERE country='China') AS T1
    WHERE city.country_id = T1.country_id) AS T2
  WHERE address.city_id = T2.city_id) AS T3
WHERE customer.address_id = T3.address_id
```

addresses in
those cities

cities in China

- The structure is like an onion.

Exercises

You must use subqueries for the following questions. Try your best to use as many subqueries as you can.

1. Find the films (title) played by Zero Cage.
2. Find the films (title) rented by George Linton.
3. Find the customers (name) who have rented some action (category) films.
4. Find the id of the films which have the lowest rental rate.
5. Find the actors who have played a same film with Bolger (the last name of an actor).
6. Count the number of customers who have not rented any film. You have to use OUTER JOIN, aggregation, and subqueries.

Save your queries in a txt file. Rename it as “COMP3013 Lab8 ####.txt”, where “####” is your student ID. And submit it on iSpace. The DDL is 24 hours after the lab.